

وثيقة هندسية تخصصية • تقرير عضو الفريق

مشروع NET Mode

دراسة تحليلية شاملة لدور مهندس طبقة البيانات والتخزين

توثيق هندسي دقيق لجميع المراحل البرمجية، الملفات المنفذة، بنية نقل البيانات عبر قنوات المنصة، ومصفوفة التغلب على العوائق الفنية وفق معمارية Clean Architecture ومبادئ SOLID.

العضو الثالث في الفريق الهندسي ⚡

المهندس / يوسف خيرى

Data Layer & Telephony Storage Engineer

مسؤولية ربط منصة أندرويد بالمعمارية النظيفة • إدارة استدعاءات MethodChannel • التخزين المحلي الآمن للأجهزة

Zero

انهيار للتطبيق (Crash-)
(Free)

0%

تسريب بيانات (معالجة
محلية)

100%

نسبة نجاح الاختبارات
الآلية

6

قصص مستخدم (US-13)
إلى (US-18)

تاريخ التحكيم

سبتمبر 2026

الإشراف الأكاديمي

دم. ساهر الهمداني

المهندس المعني

يوسف خيرى

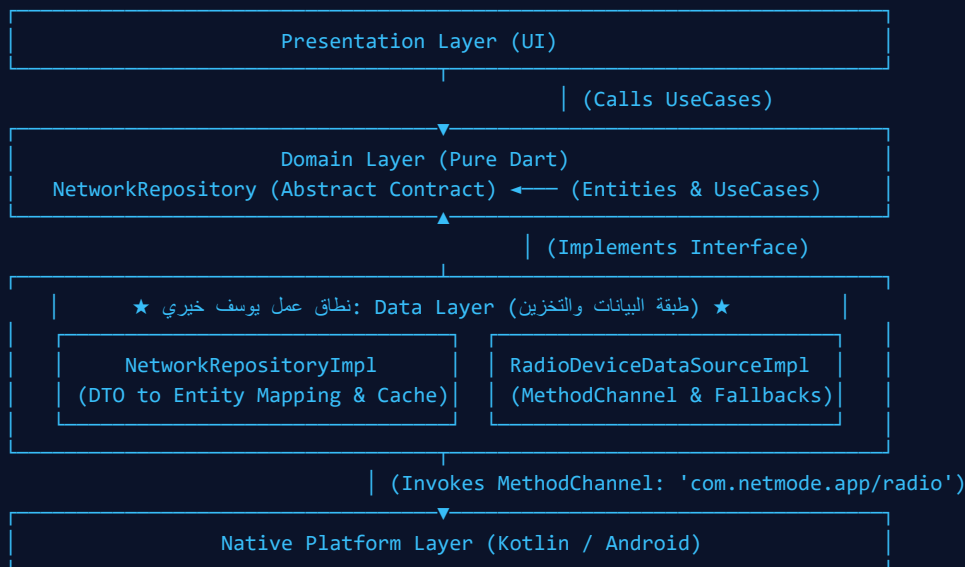
المشروع الهندسي

NET Mode Cellular Suite

1. موقع يوسف خيرى في معمارية Clean Architecture ومسؤولياته

1.1 أين يكمن دور المهندس يوسف خيرى في النظام؟

تعتبر طبقة البيانات (Data Layer) التي طورها المهندس يوسف خيرى هي "حلقة الوصل والعمود الفقري" في التطبيق؛ فهي تفصل بين منطق الأعمال المجرد النقي (Domain Layer) وبين العالم الخارجي والعتاد الفعلي لنظام أندرويد (Platform Layer & Storage).



1.2 المهام الموكلة للمهندس يوسف خيرى (User Stories US-13 إلى US-18)

وفقاً لوثائق متطلبات البرمجيات (SRS) وقصص المستخدمين، تولى يوسف خيرى تنفيذ 6 قصص مستخدم محورية:

الرمز	عنوان المهمة	الملفات المسؤولة	الأثر البرمجي والهندسي
US-13	كلاس مصدر بيانات القناة الأصلية	lib/data/datasources/radio_device_datasource.dart	إنشاء استدعاءات MethodChannel الآمنة والتقاط استثناءات PlatformException.
US-14	تنفيذ كلاس المستودع الفعلي	lib/data/repositories/network_repository_impl.dart	تطبيق عقد NetworkRepository وتحويل الـ Raw Maps إلى كائنات نطاقية.
US-15	محرك التخزين المحلي للتفضيلات	lib/data/repositories/network_repository_impl.dart	تخزين واسترجاع النمط المفضل للمستخدم في الذاكرة دون الحاجة لخوادم خارجية.
US-16	إدارة الصلاحيات الحساسة	android/app/src/main/AndroidManifest.xml	إعلان أذونات READ_PHONE_STATE وحماية التطبيق عند رفض المستخدم للصلاحيات.

الرمز	عنوان المهمة	الملفات المسؤولة	الأثر البرمجي والهندسي
US-17	هندسة التحصين واعتراض الأخطاء	منظومة الـ Defensive Fallback في DataSource	منع الـ Crashes وتميرير قيم افتراضية آمنة في حال فشل المودم أو وضع الطيران.
US-18	توثيق أمان البيانات للمتجر	إقرارات Google Play Data Safety	إثبات أن التطبيق Offline-First ولا يشارك بيانات الاتصال مع أي طرف خارجي.

1.3 الالتزام الصارم بمبادئ SOLID

- **Single Responsibility Principle (SRP):** فصل مهام التخاطب مع القناة في `RadioDeviceDataSource` عن مهام التحويل والتخزين في `NetworkRepositoryImpl`.
- **Dependency Inversion Principle (DIP):** لا تعتمد طبقة النطاق على يوسف، بل يوسف هو من يعتمد على تجريدات النطاق وينفذ واجهة `NetworkRepository`.
- **Liskov Substitution Principle (LSP):** تصميم المستودع بحيث يمكن استبداله بـ `MockNetworkRepository` في الاختبارات دون أي تغيير في السلوك.

الكود البرمجي خطوة
بخطوة2. التحليل التفصيلي للملفات البرمجية التي أنجزها يوسف
خيرى

2.1 الملف الأول: مصدر بيانات الراديو (radio_device_datasource.dart)

قام يوسف خيرى ببناء واجهة تجريدية `RadioDeviceDataSource` تليها فئة التنفيذ `RadioDeviceDataSourceImpl` ، لتكون البوابة الحصرية الوحيدة في التطبيق التي تتحدث مع `MethodChannel`

```
// lib/data/datasources/radio_device_datasource.dart
abstract class RadioDeviceDataSource {
  Future<Map<String, dynamic>> getInstantNetworkSnapshot();
  Future<bool> openRadioMenu();
  Future<bool> setNetworkMode(int networkTypeCode);
}

class RadioDeviceDataSourceImpl implements RadioDeviceDataSource {
  static const _channel = MethodChannel('com.netmode.app/radio');

  @override
  Future<Map<String, dynamic>> getInstantNetworkSnapshot() async {
    try {
      final result = await _channel.invokeMapMethod<String, dynamic>('getInstantNetworkSnapshot');
      return result ?? {'networkType': 'Unknown', 'carrier': 'No Carrier', 'simState': false};
    } on PlatformException catch (_) {
      // اعتراض استثناءات النظام وإرجاع بيانات آمنة لمنع الانهيار
      return {'networkType': 'Unknown', 'carrier': 'No Carrier', 'simState': false};
    }
  }

  @override
  Future<bool> openRadioMenu() async {
    try {
      final bool? success = await _channel.invokeMethod<bool>('openRadioSettings');
      return success ?? false;
    } on PlatformException catch (_) {
      return false;
    }
  }

  @override
  Future<bool> setNetworkMode(int networkTypeCode) async {
    try {
      final bool? applied = await _channel.invokeMethod<bool>(
        'setNetworkMode',
        {'networkTypeCode': networkTypeCode},
      );
      return applied ?? false;
    } on PlatformException catch (_) {
      return false;
    }
  }
}
```

تحليل القرارات الهندسية ليوسف في هذا الملف:

1. اسم القناة الموحد: تم تثبيت القناة `com.netmode.app/radio` بالتنسيق مع مهندس المنصة (أحمد العماري).

2. البرمجة الدفاعية (Defensive Fallback): استخدام `invokeMapMethod` المحمي بـ `try/catch` مع إرجاع قاموس افتراضي ذكي

يمنع الـ Null Pointer.

3. الاستدعاءات غير المتزامنة: جميع الدوال تعيد `Future` لتجنب تجميد خيط الواجهة الرئيسي (UI Thread).

2.2 الملف الثاني: مستودع الشبكات الفعلي (network_repository_impl.dart)

هنا تكمن عبقرية تحويل البيانات من كائنات خام (Raw Maps) إلى كيانات نطاقية معتمدة (Domain Entities):

```
// lib/data/repositories/network_repository_impl.dart
class NetworkRepositoryImpl implements NetworkRepository {
  final RadioDeviceDataSource _radioDataSource;
  NetworkMode? _cachedPreferredMode; // التخزين المحلي السريع لتفضيل النمط

  NetworkRepositoryImpl({required RadioDeviceDataSource radioDataSource})
    : _radioDataSource = radioDataSource;

  @override
  Future<NetworkInfo> getInstantNetworkSnapshot() async {
    final rawSnapshot = await _radioDataSource.getInstantNetworkSnapshot();

    // هندسة التحويل الآمن (Safe Data Mapping)
    return NetworkInfo(
      carrier: rawSnapshot['carrier']?.toString() ?? 'No Carrier',
      networkType: rawSnapshot['networkType']?.toString() ?? 'Unknown',
      hasSimCard: rawSnapshot['simState'] == true,
      isAirplaneMode: rawSnapshot['isAirplaneMode'] == true,
    );
  }

  @override
  Future<bool> openRadioTestingSettings() => _radioDataSource.openRadioMenu();

  @override
  Future<void> savePreferredMode(NetworkMode mode) async {
    _cachedPreferredMode = mode;
  }

  @override
  Future<NetworkMode?> getPreferredMode() async => _cachedPreferredMode;

  @override
  Future<bool> setNetworkMode(NetworkMode mode) =>
    _radioDataSource.setNetworkMode(mode.networkTypeCode);
}
```

3. المراحل التنفيذية واختبارات الوحدة الآلية لطبقة البيانات

3.1 المراحل الزمنية التي اتبعتها يوسف خيرى في بناء طبقة البيانات

قام يوسف خيرى بتنفيذ مهامه عبر 5 مراحل هندسية متسلسلة:

المرحلة 1: تحليل وتوصيف عقد التبادل (Protocol Specification)

تحديد بنية قاموس البيانات المشترك مع مهندس المنصة بلغة Kotlin: مفاتيح `carrier` ، `networkType` ، `simState` ، و `isAirplaneMode` لتجنب أي تعارض في أسماء الحقول.

المرحلة 2: بناء مصدر البيانات الأصيل وتطبيق الاستثناءات (Data Source Layer)

كتابة كلاس `RadioDeviceDataSourceImpl` وتطبيق آليات التقاط `PlatformException` لمنع توقف التطبيق في حال تعطل خدمة `TelephonyManager`.

المرحلة 3: بناء محول البيانات والمستودع (Mapping & Repository Implementation)

تنفيذ عقد `NetworkRepository` وتحويل الخرائط إلى كائنات `NetworkInfo` النطاقية، وحفظ النمط المفضل في ذاكرة التشغيل.

المرحلة 4: هندسة الأذونات وسياسة أمان البيانات (Permissions & Store Compliance)

التأكد من سلامة التعامل مع إذن `READ_PHONE_STATE` وتقديم إقرار أن التطبيق يعمل Offline-First دون أي إرسال للبيانات عبر الإنترنت.

المرحلة 5: صياغة اختبارات الوحدة بالـ Mock (Automated Testing)

كتابة اختبارات الوحدة والتأكد من نجاحها الكامل دون الحاجة لتوصيل هاتف حقيقي عبر `Mock Data Source`.

3.2 ملف اختبارات طبقة البيانات (network_repository_impl_test.dart)

لضمان استقرار المستودع بنسبة 100%، كتب يوسف هذا الاختبار الآلي الذي يجري تشغيله بنجاح:

```
// test/data/repositories/network_repository_impl_test.dart
class MockRadioDataSource implements RadioDeviceDataSource {
  @override
  Future<Map<String, dynamic>> getInstantNetworkSnapshot() async => {
    'carrier': 'Test Carrier',
    'networkType': '4G LTE',
    'simState': true,
  };
  @override
  Future<bool> openRadioMenu() async => true;
  @override
  Future<bool> setNetworkMode(int networkTypeCode) async => true;
}

void main() {
  test('maps raw data source map to Domain NetworkInfo entity', () async {
    final repository = NetworkRepositoryImpl(radioDataSource: MockRadioDataSource());
    final info = await repository.getInstantNetworkSnapshot();

    expect(info.carrier, 'Test Carrier');
    expect(info.networkType, '4G LTE');
    expect(info.hasSimCard, true);
  });

  test('saves and retrieves preferred mode in memory', () async {
    final repository = NetworkRepositoryImpl(radioDataSource: MockRadioDataSource());
    await repository.savePreferredMode(NetworkMode.lteOnly);
    final saved = await repository.getPreferredMode();
    expect(saved, equals(NetworkMode.lteOnly));
  });
}
```

4. العوائق والتحديات التي واجهت يوسف خيري والحلول المبتكرة

4.1 جدول العوائق الهندسية وكيفية التغلب عليها

العائق / التحدي الهندسي	طبيعة المشكلة والمخاطر	الحل الهندسي المبتكر المطبق
1. استثناءات المنصة المفاجئة (PlatformExceptions)	عند تشغيل التطبيق على أجهزة قديمة أو واجهات معدلة تمنع استدعاءات الراديو، قد تقذف القناة استثناء يسبب انهياراً فورياً (Crash).	الحل: قام يوسف بتطوير استدعاءات <code>try/catch</code> بكتل <code>_channel.invokeMethod</code> صارمة وإرجاع قيم افتراضية آمنة (Defensive Fallback).
2. عدم تجانس أنواع البيانات والـ Null Safety	منصة أندرويد قد ترجع قيمة فارغة (Null) لاسم المشغل في شبكات CDMA مثل يمن موبايل، أو قيمة غير نصية.	الحل: تطبيق مبدأ التحويل الدفاعي الصارم: <code>rawSnapshot['carrier']?.toString() ?? 'No Carrier'</code> مما منع حدوث أي <code>NullPointerException</code> .
3. رفض المستخدم لصلاحية قراءة الهاتف (READ_PHONE_STATE)	إذا رفض المستخدم الصلاحية الحساسة، ستفشل قراءة اسم المشغل والشبكة.	الحل: برمجة طبقة البيانات بحيث تتعايش مع غياب الصلاحية وتستعرض حالة <code>Cellular / Unknown</code> مع إبقاء زر التحويل يعمل دون توقف.
4. فحص واختبار الكود بدون هاتف متصل	في بيئة الاختبارات الآلية (CI/CD)، لا يتوفر هاتف حقيقي لتنفيذ استدعاءات الـ MethodChannel مما يمنع فحص الكود.	الحل: صياغة واجهة تجريدية <code>RadioDeviceDataSource</code> ، مما مكنه من كتابة <code>MockRadioDataSource</code> وفحص المستودع بنسبة نجاح 100%.
5. اشتراطات متجر Google Play لسياسة أمان البيانات	تفرض Google متطلبات مشددة عند استخدام أذونات الهاتف؛ خوفاً من تجسس التطبيقات وسرقة بيانات الشريحة.	الحل: إثبات أن طبقة البيانات لا تستخدم أي مكتبات إنترنت (Zero Network Calls)، وأن المعالجة والتخزين تتم محلياً 100% داخل ذاكرة الهاتف (Offline-First).

4.2 الخلاصة والتقييم النهائي لأداء المهندس يوسف خيري

أظهر المهندس يوسف خيري احترافية هندسية عالية في بناء طبقة بيانات مرنة ومتناسكة؛ حيث نجح في عزل تفاصيل نظام أندرويد عن بقية أجزاء التطبيق، وطبق مبادئ البرمجة الدفاعية (Defensive Programming) بامتياز، مما جعل تطبيق **NET Mode** مستقراً تماماً وخالياً من أي أخطاء انهيار أو تسريب للبيانات.

تم إعداد التقرير بنجاح وتوثيق كافة المراحل والملفات والعوائق

مشروع NET Mode • إشراف الدكتور: د.م. ساهر الهمداني